Report: Cardio Risk Prediction

1. Exploratory Data Analysis (EDA)

Your report should present a clear and thorough EDA of the training data, including data cleaning steps, univariate and bivariate analyses, numerical feature distributions and outliers, correlation analysis, class balance checks, target likelihood analysis for categorical variables, and relationships between numerical features and the target.

Use appropriate visualisations with meaningful interpretations that link back to the business objective.

Answer:

Section 2 Data Cleaning

Section 2.1 Identify and handle redundant or invalid/illogical physiological values

Section 2.2.1 Check Statistical summary of the data

The *.describe()* in-built function is used to get a summary of the dataset to identify any missing or invalid values that are present.

```
# Check the statistical summary of the data
df.describe()
```

	age	gender	height	weight	ap_hi	ap_lo	cholesterol	gluc	smoke	alco	active	cardio
count	70000.000000	70000.000000	70000.000000	70000.000000	70000.000000	70000.000000	70000.000000	70000.000000	70000.000000	70000.000000	70000.000000	70000.000000
mean	19468.865814	0.349571	164.359229	74.205690	128.817286	96.630414	0.366871	0.226457	0.088129	0.053771	0.803729	0.499700
std	2467.251667	0.476838	8.210126	14.395757	154.011419	188.472530	0.680250	0.572270	0.283484	0.225568	0.397179	0.500003
min	10798.000000	0.000000	55.000000	10.000000	-150.000000	-70.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	17664.000000	0.000000	159.000000	65.000000	120.000000	80.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000
50%	19703.000000	0.000000	165.000000	72.000000	120.000000	80.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000
75%	21327.000000	1.000000	170.000000	82.000000	140.000000	90.000000	1.000000	0.000000	0.000000	0.000000	1.000000	1.000000
max	23713.000000	1.000000	250.000000	200.000000	16020.000000	11000.000000	2.000000	2.000000	1.000000	1.000000	1.000000	1.000000

From the output above, we see that the dataset has 70,000 records in it. The general summary statistics are "*Count*" (number of non-missing values), "*Mean*" (average value), "*Std*" (Standard Deviation/spread of data), "*min*" (Minimum; smallest value), "*25%*" (first quartile), "*50%*" (median), "*75%*" (third quartile), "*max*" (Maximum; largest value).

In the "Height" column, we see that there are unrealistic values like 55cm and 250cm which indicates outliers or data entry error and should be removed or capped. Similarly, in the

“Weight” column, we see that there is another unrealistic range from 10kg (impossible for adults to weigh) to 200kg (extreme). Hence, suggesting that outliers exist in the dataset and Inter-Quartile Range (IQR) filtering or clipping can be done to remove it.

We also see that lifestyle and the target variable have been correctly encoded and the dataset is balanced which is suitable to build models.

Section 2.1.2 Handle rows with invalid/illogical values

The rows that were identified having illogical or invalid values are: “ap_hi”, “ap_lo”, “height” and “weight”. To remove them, the following steps have been taken:

1) The systolic and diastolic blood pressure (BP) columns (“ap_hi”, “ap_lo”) filters the dataset and keeps rows where the values are between 30 and 300. This means that any negative values, extremely high/impossible values and unrealistic values are removed.
2) Similarly, for the “height” column, only rows where the height is in between 100 and 250 in centimeters are kept. Beyond these values, it would be deemed unrealistic or impossible values. This is done using the standard Boolean filter.
3) The same is done for the “weight” column where rows with weights between 30 and 250 are kept using the standard Boolean filter.
4) The first 5 rows for “height” and “weight” columns are displayed using the *.head()* function.

```
df = df[(df['ap_hi'].between(30,300)) & (df['ap_lo'].between(30,300))]

df = df[(df['height'] > 100) & (df['height'] < 250)]
df = df[(df['weight'] > 30) & (df['weight'] < 250)]

print(df['height'].head())
print(df['weight'].head())
```

```
0    168.0
1    156.0
2    165.0
3    169.0
4    156.0
Name: height, dtype: float64
0    62.0
1    85.0
2    64.0
3    82.0
4    56.0
Name: weight, dtype: float64
```

From the output above, we see that the values for height and weight fall between the range that was used to filter out illogical values. Hence, it can be said that the data has been

cleaned correctly. We also see that the datatype for both columns are float/decimal point numbers.

Section 2.1.3 Modify the representation of patient age and height

The code below is used to represent age and height in a much easier way to understand in the context of healthcare. The "age" column is currently in the form of days, which is difficult to understand. Similarly, "height" is converted to meters.

1) The "age" column is converted to years by dividing the age that is in terms of days by 365 (age/365), as 1 year = 365 days.
2) The result is then rounded off to 1 decimal place with the help of the *round()* in-built function. This makes it easier to read and understand the age of each patient in the dataset.
3) Similarly, the "height" column is converted from centimeters (cm) to meters (m) with the formula: (height/100).
4) The first 5 rows of both columns, "height" and "age" are displayed using the *.head()* function

```
df['age'] = round(df['age'] / 365, 1)
df['height'] = df['height'] / 100

print(df['age'].head(), df['height'].head())
```

```
0    50.4
1    55.4
2    51.7
3    48.3
4    47.9
Name: age, dtype: float64
0    1.68
1    1.56
2    1.65
3    1.69
4    1.56
Name: height, dtype: float64
```

From the output above, we see that the "age" and "height" have been converted successfully as intended. This step will then help the model to understand the age and height of the patients in the dataset during training.

Section 2.2 Fix data types

Section 2.2.1 Review and fix the data types of all columns

The code below is used to fix the data types of some columns where they were stored as numbers but are categorical.

1) A list of the actual categorical columns is created as they do not have continuous numbers. These columns are: "gender", "cholesterol", "gluc", "smoke", "alco", "active", "cardio".

2) The for loop is then used to loop through each column in the list.
3) The columns are then converted to categorical datatype (category) using the *.astype()* function.
4) The .dtypes function is then used to show the datatype of each column present in the dataset and to verify if the highlighted columns have been converted correctly.

```
# Fix DataTypes of the categorical columns with incorrect DataTypes
cat_cols = ['gender','cholesterol','gluc','smoke','alco','active','cardio']
for col in cat_cols:
    df[col] = df[col].astype('category')
```

```
df.dtypes
```

	0
age	float64
gender	category
height	float64
weight	float64
ap_hi	float64
ap_lo	float64
cholesterol	category
gluc	category
smoke	category
alco	category
active	category
cardio	category

dtype: object

As seen in the output on the left, the data types for each column have been fixed and displayed using the *.dtypes* in-built attribute.

We see that some of the columns (eg: “age”, “height”, “weight”, “ap_hi” and “ap_low”) have been changed to “float” datatype as they are continuous variables.

On the other hand, the remaining columns were changed to “category” datatype as they could have been binary/discrete variables.

Section 3 EDA

Section 3.1 Univariate Analysis

Section 3.1.1 Visualise the numerical features

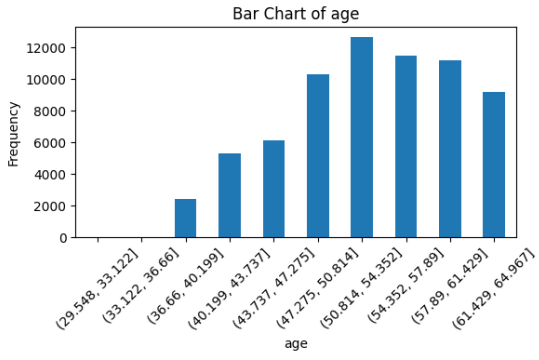
The code below plots all the numerical columns to visualise and understand how the values are distributed within the dataset.

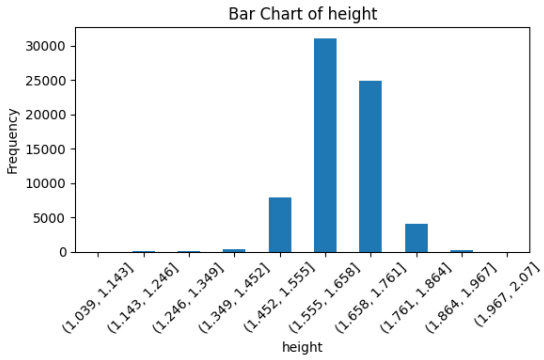
1) The first line of code focuses on the columns present in the dataset using the *.columns* method.
2) Out of all the columns, the numerical columns are selected from the dataset using the *.select_dtypes()* function. The *“include”* attribute within the function includes any column with the datatype of integer or float.

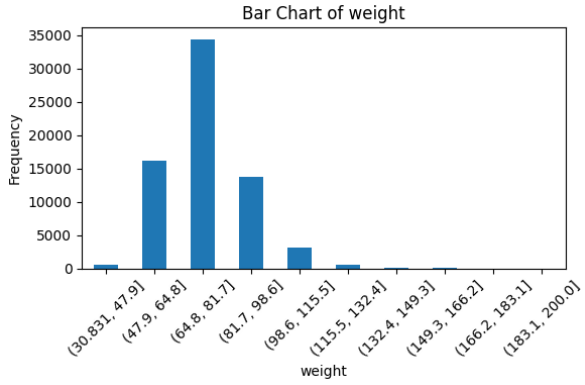
The numerical columns which will be selected are: “age”, “height”, “weight”, “ap_hi”, and “ap_lo”.

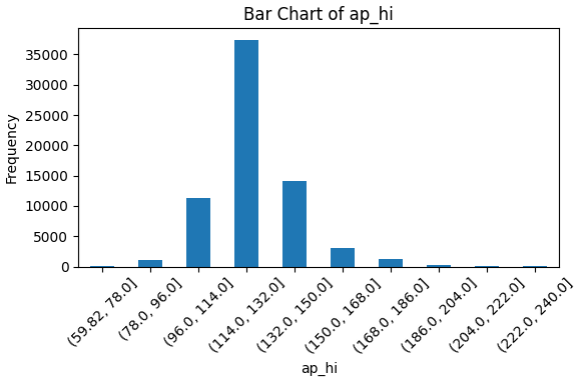
3) For each column in “num_cols”, a bar plot is created to show the distribution of them, using the *.figure()* function which plots a bar plot by default.
4) Bins are created using *.cut()* function where the data is divided into 10 equal ranges (eg: 140-145, 145-150, etc). Then, the *.value_counts()* function is used to calculate how many values fall into each bin/range. Finally, the *.sort_index()* function sorts these bins in order.
5) The bar plots are in a tight layout by adjusting spacing and preventing overlapping, using *.tight_layout()*. Along with that, the *.xticks(rotation=45)* rotates the labels on the x-axis to make them more readable.

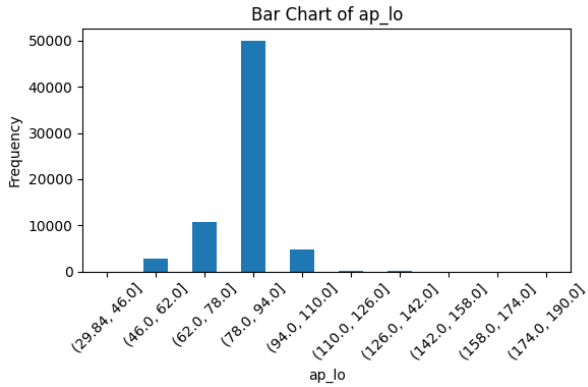
```
# Plot all the numerical columns to understand their distribution
num_cols = df.select_dtypes(include=['int64', 'float64']).columns

for col in num_cols:
    plt.figure(figsize=(6,4))
    binned = pd.cut(df[col], bins=10).value_counts().sort_index()
    binned.plot(kind='bar')
    plt.title(f'Bar Chart of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```











We see that there are 5 graphs that have been given as output to the code, as shown above.

The insights for each chart are given as below:

- From the “age” bar chat, we see that the bars increase from left to middle and peak around 50-60 years before slightly decreasing. This suggests that most of the people in the dataset (who have or do not have any cardio risk issues) are middle-aged with the range being between 45-60 years.
- From the “height” bar chart, we can see that most of the values are between 155-175 cm with the highest bar plot being 160-165cm. There are also very few extreme heights. This shows that the population has typical average adult height and the distribution looks normal.
- From the “weight” bar chart, the peak is around 65-80kg which suggests that most of the people are between normal to overweight range and is common. Some people weigh between 80-100kg and few above 120kg. The higher the weight, the increase in risk of heart disease.
- From the “ap_hi” bar chart, we can see most of the values lying around 110-130 and 120-140 with very few extremes. This suggests that most of the people in the dataset

have a normal or slightly high BP. The range 120-140 could indicate pre-hypertension or stage 1 hypertension.

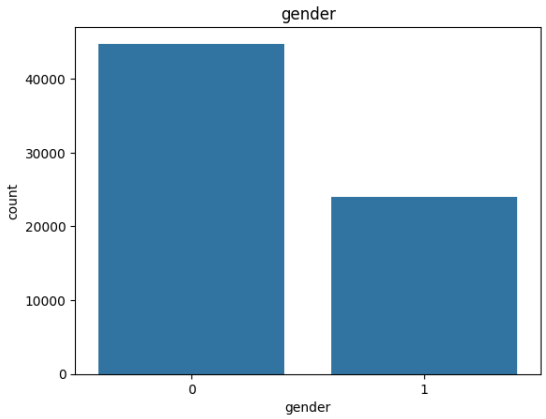
- Similar to "ap_hi", we see the "ap_lo" bar chart with majority of the population falling between the range of 78-94, while the range of data points is between 62 and 94 indicating that the average is around 80 (which is considered as normal).

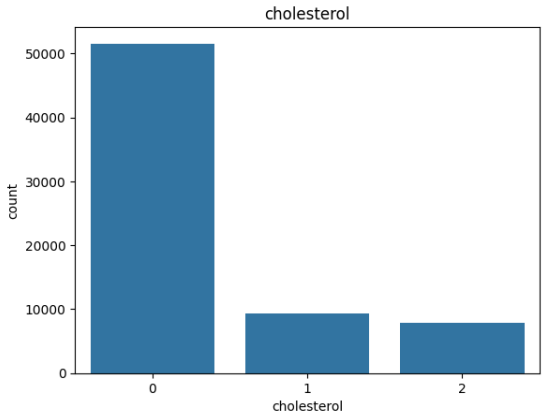
Section 3.1.2 Visualise the categorical features

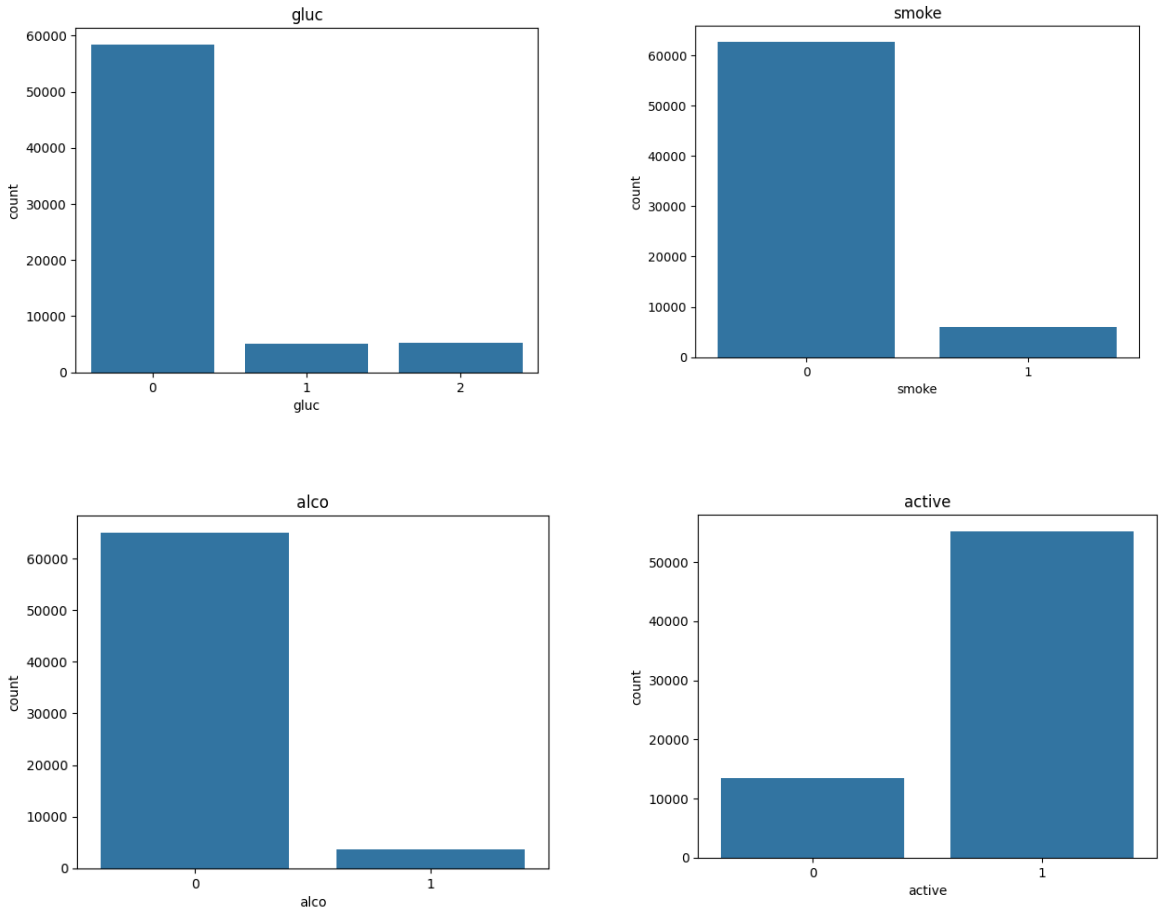
The code below selects and plots the categorical columns, similar to the previous section.

1) The categorical datatype features are selected using the *.select_dtypes()* function with the *include* attribute being "category". To select only the columns, the *.columns* method is used.
2) For each column in "cat_cols", charts are created to show the distribution of them, using the *.figure()* function.
3) Using the seaborn package's *.countplot()* function, count plots are created for each categorical column to show the number of records for each category. Count plots are bar charts for categories, which count how many times each category appears in the dataset.

```
cat_cols = df.select_dtypes(include='category').columns
for col in cat_cols:
    sns.countplot(x=col, data=df)
    plt.title(col)
    plt.show()
```







The outputs above show the countplots for each of the categorical columns. Detailed explanations are given below.

- From the “gender” countplot, we see that there is a significant imbalance where there are more females (represented by 0) than males (represented by 1). This could suggest that any conclusions from the dataset would be more robust for the females (since they are in the majority).
- From the “smoke” countplot, we see that most of the participants are non-smokers (represented by 0) and only a small percentage being smokers (represented by 1). This suggests that although smoking is a major risk factor for heart disease, this dataset is mostly non-smokers which could mean that this factor may not be that significant.
- From the “alco” countplot, we see that alcohol consumption (represented by 1) is less frequent than smoking in the group, meaning that the population is generally sober and do not drink.

Assumption: The participants have not under-reported/misreported that they do not drink as frequently.

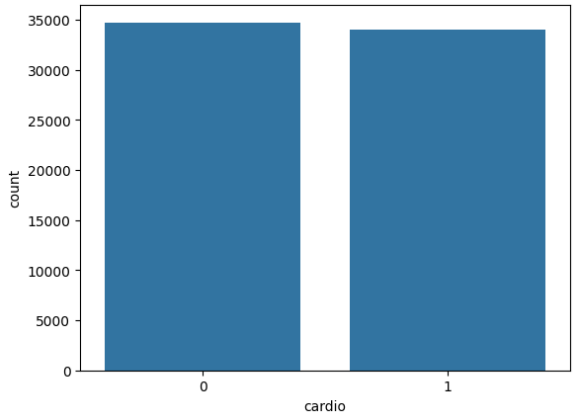
- From the "active" countplot, we see that most of the participants, approximately 55000, are physically active (represented by 1) and a much smaller group of participants, approximately 130000, are physically inactive (represented by 0). This suggests that the participants perceive themselves to be active, which is a preventive measure against cardio-related diseases.
- From the "cholesterol" countplot, we see that there are 3 categories: 0 (Normal), 1 (Above normal), 2 (Well above normal). We can see that most people have normal cholesterol but there is a visible population of participants in the other categories. An insight into this could be that there are enough high-cholestrol cases to perform a meaningful analysis on how lipids correlate with heart disease.
- From the "gluc" countplot, we see that this also has 3 categories: 0 (Normal), 1 (Above normal) and 2 (Well above normal). Like cholesterol, it is observed that most individuals have normal glucose levels and the number of people with elevated glucose levels are lower than those with elevated cholesterol levels.

Section 3.1.3 Check class distribution of target feature

The code below is used to check the distribution of values of the target feature (cardio).

1) A count plot is created for the "cardio" column using the *.countplot()* function in Seaborn package. The "x" attribute is the target feature ("cardio") and the "data" attribute is the entire dataset ("df").
2) The .value_counts(normalize=True) function is used to count the number of occurrences of values in the column. The *normalize* attribute converts the counts into percentage/proportion using the formula: (total count/total number of rows)

```
# Class distribution of positive and negative classes
sns.countplot(x='cardio', data=df)
plt.show()

df['cardio'].value_counts(normalize=True)
```



cardio	
0	0.505062
1	0.494938

dtype: float64

From the output above, we can first interpret that 0 represents the number of people without heart disease and 1 represents the number of people with heart diseases. We can also see that the dataset is relatively balanced and evenly distributed with those without cardiovascular disease being 50.5% and those with cardiovascular disease being 49.4%. A balanced dataset is good for training models as it avoids any bias from forming.

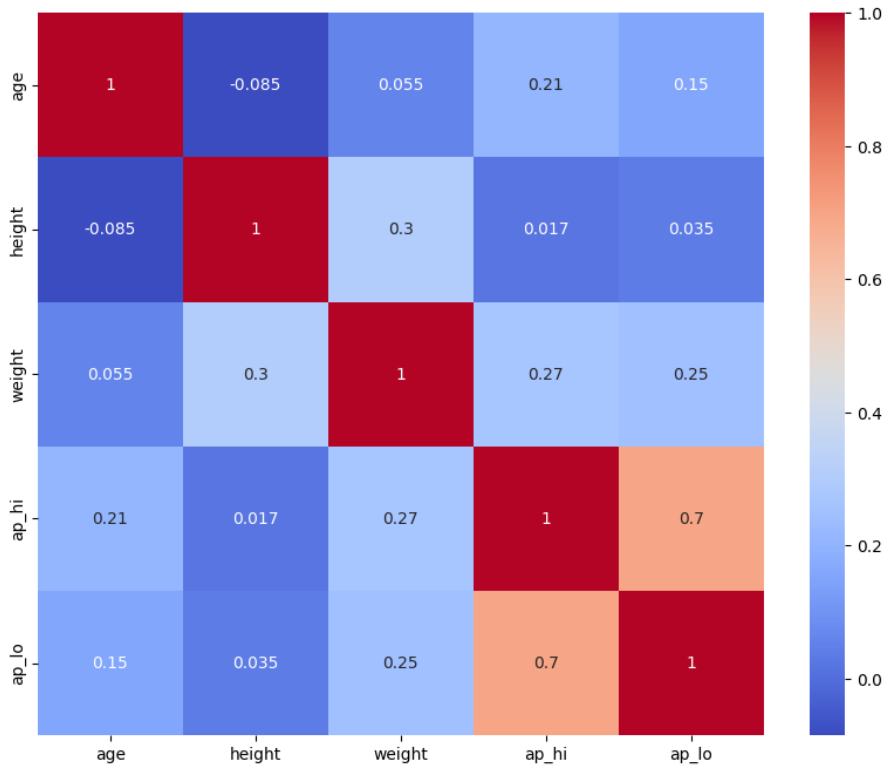
Section 3.2 Correlation Analysis

Section 3.2.1 Visualise the correlation among numerical features

To visualize a correlation between the numerical features, the best visualization to use is the correlation heatmap.

1) A plot of size 10x8 is created using *.figure(figsize=(10,8))* method to make the heatmap larger and readable.
2) A heatmap is created using the *.heatmap()* function in the Seaborn package. The numerical columns are selected using df[num_cols] – which are: “age”, “height”, “weight”, “ap_hi”, “ap_lo”.
3) Then, the .corr() function calculates the correlation matrix to show how strongly related 2 variables are.
   The correlation values can range from -1 to +1 where +1 means a strong positive relation, 0 means no relation and -1 means strong negative relation.
4) The “annot=True” parameter within the in-built function is used to display the correlation values in the boxes.
   The “cmap=’coolwarm’” parameter within the same function sets a colour scheme or red, blue and white where red means a positive correlation, white means no correlation and blue means negative correlation.

```
# Plot Heatmap of the correlation matrix
plt.figure(figsize=(10,8))
sns.heatmap(df[num_cols].corr(), annot=True, cmap='coolwarm')
plt.show()
```



From the output above, we see that the strongest positive relationship amongst the numerical features is between “ap_hi” and “ap_low” with a score of 0.7. This implies that if either of these increases, the other also increases proportionally. We can also assume that these 2 features are multicollinear due to them being highly correlated with each other.

We also see that “weight” has a moderately positive correlation with “height” with a value of 0.3. This is somewhat expected as tall people tend to weigh more but it does not account for their body mass index (BMI).

Similarly, “weight” has a positive correlation with both “ap_hi” and “ap_lo” with values 0.25 and 0.27, respectively. This suggests that as weight increases, the blood pressure also tends to increase almost proportionally.

On the other hand, we see that “age” and “height” have a correlation value of -0.085, which is closer to 0. Hence, we can say that these 2 features have no correlation with each other.

Section 3.2 Bivariate Analysis

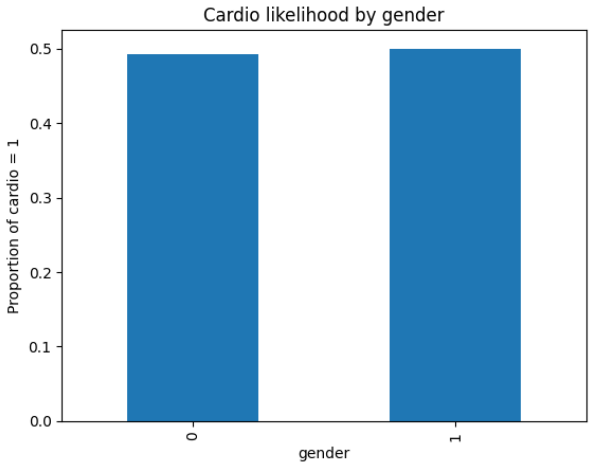
Section 3.3.1 Analyse categorical features

The code below is used to analyse the categorical features present by calculating the proportion of each category with "cardio=1" present. This helps in identifying which categorical features show clear differences in likelihood of heart disease.

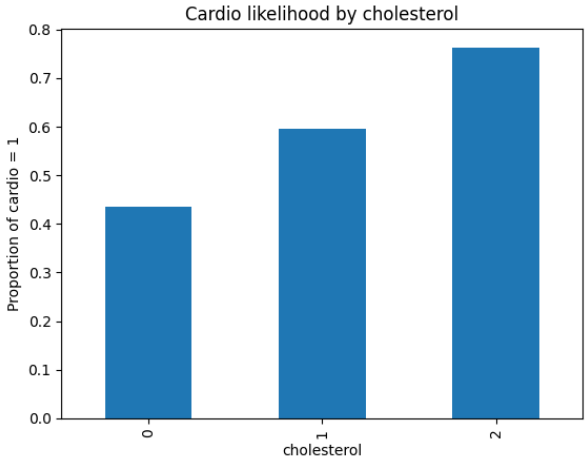
1) A function called "categoricalTarget" is created and it takes 2 inputs – the dataset and "column" for categorical columns. A "temp" dataframe is created by copying the data from the original dataset using the *.copy()* function to not modify the original data.
2) Then, the "cardio" column is converted to integer using *.astype()* as it was originally of "category" datatype and is now needed for calculation.
3) By using *.groupby()*, the data is grouped by the categorical feature and then selects the "cardio" column to calculate the average of it using *.mean()*
4) The result of calculation in step 3 is then plotted as a bar chart with the x-axis having categories and y-axis having the probability of cardio (cardio=1).
5) The for loop goes through the categorical columns and if while looping the column is "cardio", it is skipped since we cannot plot "cardio" against "cardio".

```
def categoricalTarget(data, column):
    temp = data.copy()
    temp['cardio'] = temp['cardio'].astype(int)
    res = temp.groupby(column)['cardio'].mean()
    print(res)
    res.plot(kind='bar')
    plt.title(f'Cardio likelihood by {column}')
    plt.ylabel("Proportion of cardio = 1")
    plt.show()

for col in cat_cols:
    if col != 'cardio':
        categoricalTarget(df, col)
```

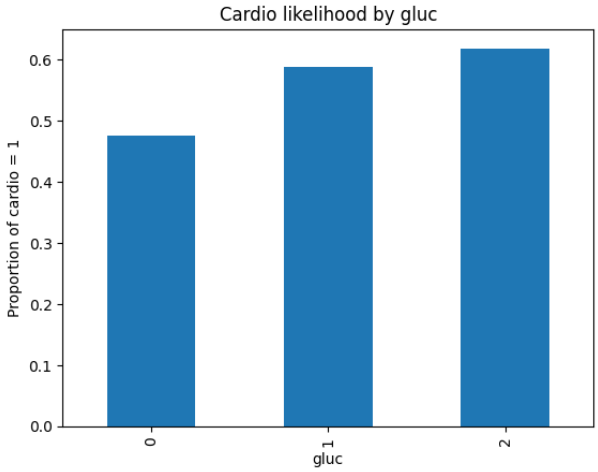
```
gender
0    0.492260
1    0.499937
Name: cardio, dtype: float64
```



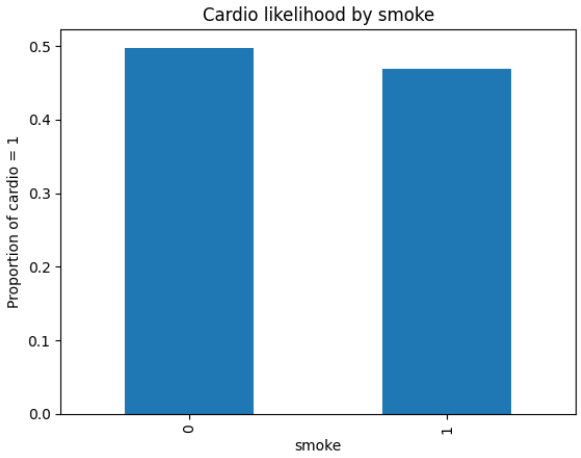

```
cholesterol
0    0.435633
1    0.596391
2    0.762908
Name: cardio, dtype: float64
```



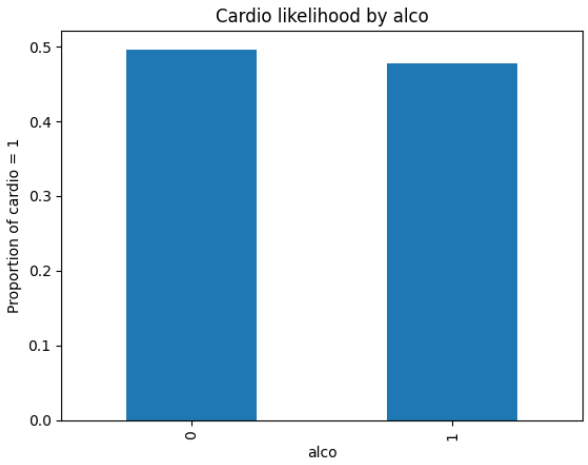

```
gluc
0    0.475716
1    0.588687
2    0.618647
Name: cardio, dtype: float64
```



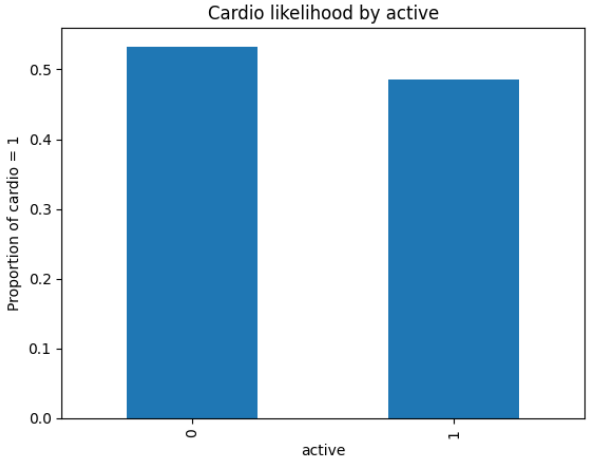
```
smoke
0    0.497472
1    0.468673
Name: cardio, dtype: float64
```




```
alco
0    0.495903
1    0.477895
Name: cardio, dtype: float64
```




```
active
0    0.532737
1    0.485686
Name: cardio, dtype: float64
```



We immediately see that if a bar is significantly higher for one category than another, that feature is a strong indicator of heart disease. From the above charts of each feature:

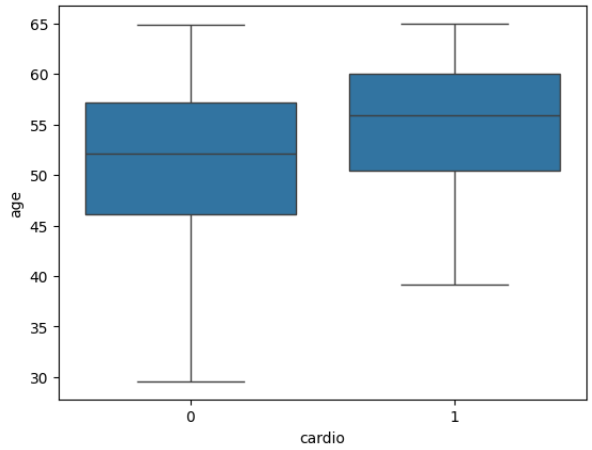
- We see that “cholesterol” is the most significant predictor in the dataset as the category moves from 0 (normal) to 2 (well above normal), there is a huge increase in likelihood of heart disease from 43% to 76%. This confirms that having high cholesterol is a critical risk factor for cardiovascular diseases.
- Similarly, we see that “gluc” (glucose) is another significant predictor in the dataset as the risk increases from 47% at normal glucose level to 61% if glucose level is well above normal.
- For the “active” bar plot, we see that those who are not active (represented by 0) have a 53% chance of getting heart disease, compared to those who are active (represented by 1) with a 48%. This shows that physical activity is a protective factor by lowering the probability of cardiovascular disease.
- From the “smoke” and “alco” plots, we see that smokers and drinkers show a lower proportion of heart disease (47% chance of getting cardiovascular disease) compared to the non-smokers and non-drinkers (49% chance of getting cardiovascular disease). However, this could be an issue of selection bias that has occurred in the dataset.
- From the “gender” plot, we see that the likelihood of contracting heart disease for either gender is almost identical – females being 49% while males being 50%. This suggests that it is a weak predictor as there is no extra information given.

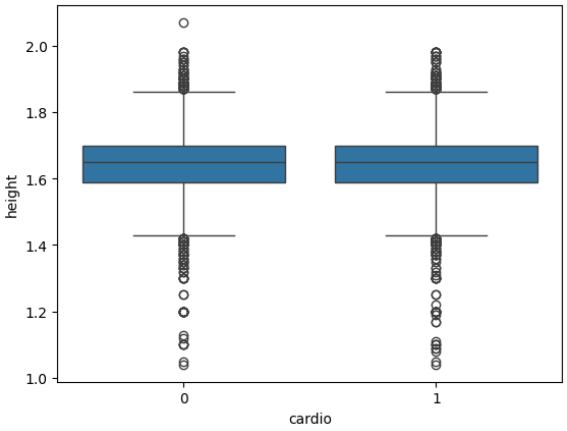
Section 3.3.2 Explore the relationships between numerical features and the target variable

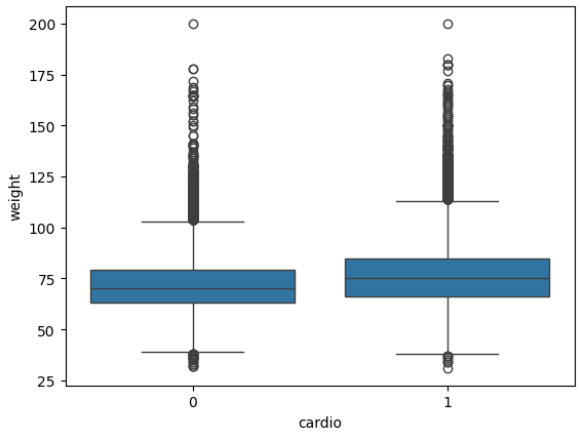
The code below is used to look at the relationships between the numerical features and their target variables to identify trends and potential interactions.

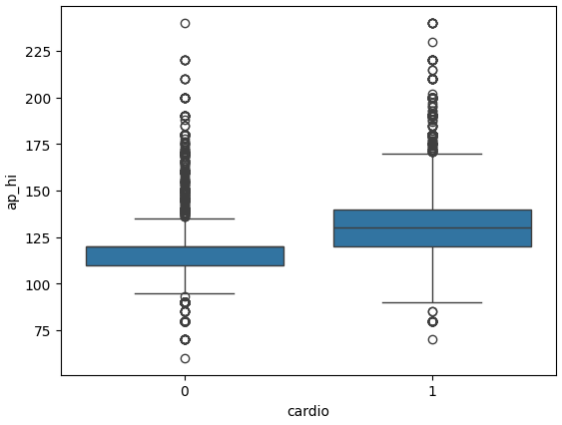
1) The for loop is used to iterate through each numerical column in the dataset.
2) A box plot is used to display the relationships between the numerical columns and the target variable with the help of *.boxplot()* function in the Seaborn package.
3) The x-axis consists of the target variable, “cardio” and the y-axis consists of each numerical column.

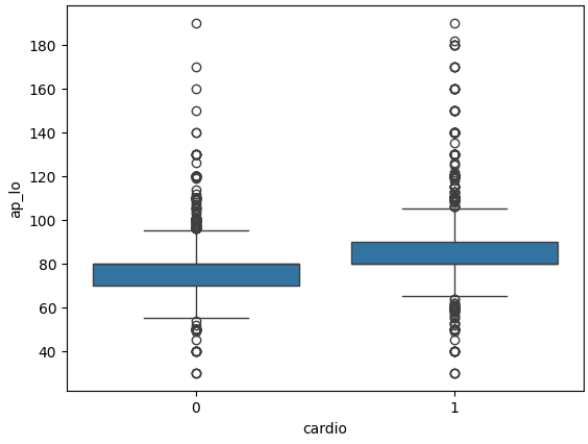
```
# Plot distribution for each numerical column with target variable
for col in num_cols:
    sns.boxplot(x='cardio', y=col, data=df)
    plt.show()
```











From the above outputs, we can interpret the following:

- From the “age vs cardio” boxplot, we see that the median age for those who have cardiovascular disease (represented by 1) is higher than that for those in the healthy

group. This suggests that it is more prevalent in the older individuals (age range: 50-60) within the dataset, while the healthy group has a much wider spread with younger people under 45 years of age.

- From the “height vs cardio” boxplot, we see that the boxes are almost identical in level and size. This suggests that height is not a prevalent factor or differentiator for heart disease in the dataset. There are also many outliers which could suggest some extreme values that are still present.
- From “weight vs cardio” boxplot, we see that the box for those with cardiovascular disease is higher than the box for those without cardiovascular disease. The median weight for the group where “cardio = 1” is around 75-80kg while the healthy group’s median is closer to 70kg. This suggests that people who weigh more tend to have a cardiovascular disease.
- From “ap_hi vs cardio” boxplot (“ap_hi” refers to Systolic blood pressure), we see that there is a very clear separation. The “cardio=0” (healthy participants) box is narrow and low while the “cardio=1” (heart disease participants) is higher and much wider. This could mean that systolic blood pressure is one of the strongest indicators of a cardiovascular disease occurring.
- Similarly, from “ap_lo vs cardio” boxplot (“ap_lo” refers to Diastolic blood pressure), we see that it is significantly higher in the diseased group. This could suggest that most healthy participants have a median diastolic blood pressure of approximately 80 while participants with cardiovascular disease have a median closer to 90.

---

2. Model Building and Rationale

Your report should clearly describe all candidate models (e.g., Decision Tree, SVM, additional models if used), the rationale for selecting them, the feature selection or engineering process, the choice of cutoffs, and justification for hyperparameters. The report should explain the full model-building process and why each modelling decision is appropriate for the data and task.

Answer:

Section 4 Train-Test Split

Section 4.1.1 Define feature and target variables

The code below is used to split the data in the dataset into 2 groups – X and y, where "X" represents the independent features while "y" represents the target variable.

1) From the entire dataset, the "cardio" column is dropped from it using the .drop() function. The "axis=1" attribute is used to ensure that the column is dropped, not the row (axis = 0).
2) The "y" variable stores the dropped "cardio" column from the dataset.

```
X = df.drop('cardio', axis=1)
y = df['cardio']
```

Section 4.1.2 Split the data into train and test sets

The data from the previous step is then split accordingly as shown below.

1) The data from the previous step – "X" and "y" is further split into "X_train", "X_test", "y_train" and "y_test".
   "X_train" = 70% of the features are used to train the model
   "X_test" = 30% of features are used to test the model
   "y_train" = Target values that are used for training the model
   "y_test" = Targest values that are used for testing the model
2) The *"test_size = 0.3"* attribute means that 30% of the data is test data while the remaining 70% of the data train data. The *"random_state=42"* attribute ensures that the same random split is done every time, else each run would give different data split.
3) The *"stratify=y"* attribute is used to keep class distribution the same in train and test data. Without it, the model may learn patterns from the data wrongly.
4) The next 4 lines of code are used to reset the row numbers of all 4 split datasets, using the *.reset_index()* function. This is done as indexes usually look messy after splitting is done.
5) The "drop=True" attribute is used to remove the old index column as it becomes a new column after resetting index. The "inplace=True" attribute modifies the original dataframe instead of creating a new one.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42, stratify=y)
```

```
X_train.reset_index(drop=True, inplace=True)
X_test.reset_index(drop=True, inplace=True)
y_train.reset_index(drop=True, inplace=True)
y_test.reset_index(drop=True, inplace=True)
```

Section 5 Feature Engineering

5.1 Create a new feature

5.1.1 Create a new feature BMI (Body Mass Index)

A new feature called “BMI” is created with the help of the “height” and “weight” columns. This is done since it is a useful predictor for cardiovascular risk, compared to height and weight. Since the features are present in only the “X” variable and the data has been split, we add the new feature to “X_train” and “X_test”.

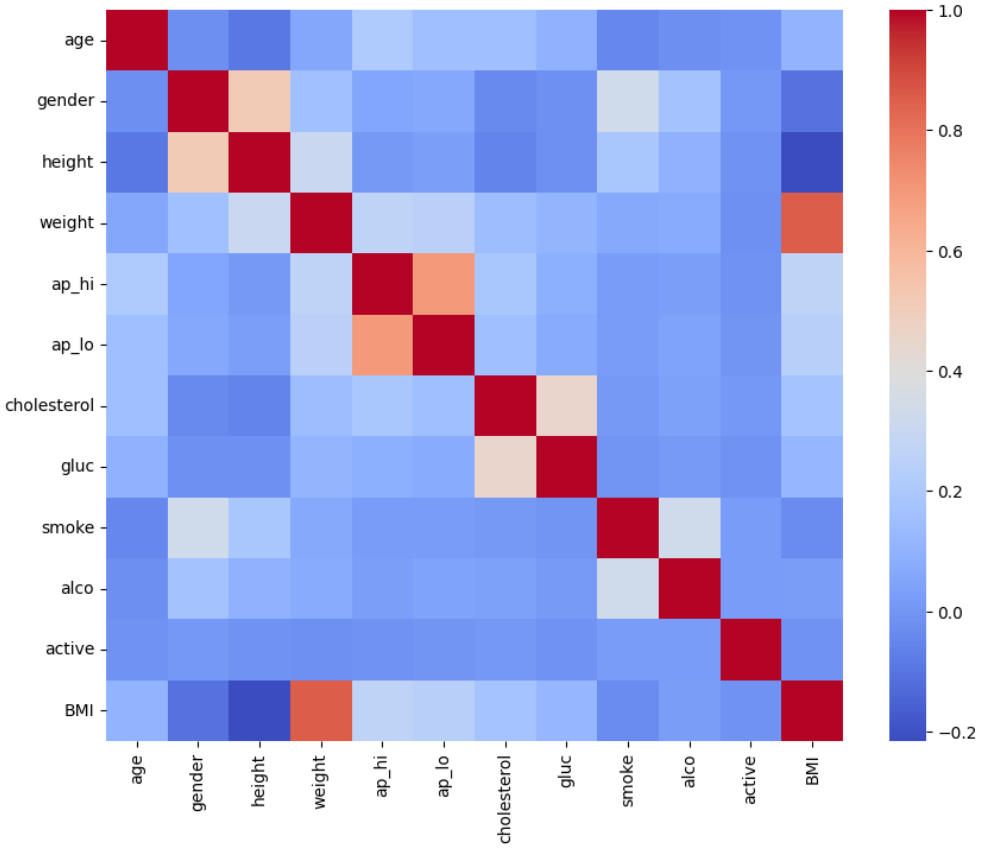
1) Firstly, calculate the BMI for each feature present in “X_train” by using the formula: weight/(height**2)
2) Like the first step, calculate the BMI for each feature present in “X_test” by using the same formula.

```
# Create a new feature 'BMI'
X_train['BMI'] = X_train['weight'] / (X_train['height']**2)
X_test['BMI'] = X_test['weight'] / (X_test['height']**2)
```

5.1.2 Perform Correlation Analysis

After adding the new BMI feature, it is wise to perform correlation analysis to understand how the new feature is correlated with the existing features and see if it is suitable for processing.

```
# Plot check correlation
plt.figure(figsize=(15,10))
sns.heatmap(X_train.corr(), cmap='coolwarm')
plt.show()
```



The output above shows:

- That “weight” and “BMI” have a strong positive correlation of value 0.8. This holds true as BMI is a direct derivation of weight.
- However, “height” and “BMI” have a negative correlation value 0.2. This could happen as the “height” is the denominator in the formula for BMI. As height increases, the BMI decreases.
- The “ap_hi” and “ap_lo” columns remain strongly correlated and they both show some correlation with BMI which reinforces the fact that BMI, age, blood pressures are quite related.

Question: Did you find any highly correlated features? What steps should you take if high correlation (>0.9) exists?

Answer:
No, there are no findings of correlation values being more than 0.9. The highest correlation values at present are 0.7 and 0.8.

If there were some highly correlated values, it would mean that there is multicollinearity present. Hence if it exists, the steps to take are:

1) Dropping a feature when 2 variables are almost identical (>= 95%). The model does not require both features; hence it would be wiser to keep the one with fewer missing values.
2) Combine both into a single score through feature engineering. An example would be the way BMI was derived from height and weight.
3) Check the Variance Inflation Factor (VIF) score. If it is more than 5 or 10, it means that it has high multicollinearity and the variable must be dropped.

5.2 Combine values in categorical columns

5.2.1 Combine low frequency categories

Some of the features like "cholesterol" and "gluc" have low-frequency categories present in the dataset. It is important to combine them so that data sparsity will reduce and the model generalization will improve.

1) In the "cholesterol" column for both "X_train" and "X_test", category 3 is merged into category 2 using the *.replace()* method so that only 2 categories remain: cholesterol = 1 and cholesterol = 2.
2) Similarly, in the "gluc" column for both "X_train" and "X_test", category 3 is merged into category 2, using the same method.
3) This step is done for both train and test data as both sets of data must have the same categories. Otherwise, the model will fail during training.

```
X_train['cholesterol'] = X_train['cholesterol'].replace({3:2})
X_test['cholesterol'] = X_test['cholesterol'].replace({3:2})

X_train['gluc'] = X_train['gluc'].replace({3:2})
X_test['gluc'] = X_test['gluc'].replace({3:2})
```

5.3 Dummy variable creation

5.3.1 Create dummy variables for categorical columns

Dummy variables for all the categorical columns have been created as the models, in the upcoming steps, require numerical input and are unable to process categorical values.

1) The categorical columns are identified in the "X_train" dataset by using *.select_dtypes('category')* and *.columns* functions.
2) The *.get_dummies()* function is used to convert the categorical columns that were identified in the previous step into numerical binary columns through One-Hot

Encoding technique. The “drop_first = True” attribute will remove the first category to avoid any dummy variable traps from occurring.

3) Dummy variables are also created for the “X_test” dataset, the same as the previous step.
4) The columns in “X_test” are aligned according to the “X_train” columns using the *.reindex()* function. This step helps in ensuring that the test data has the same categories as the ones in the train data. The “fill_value=0” attribute is used in case a column is missing in test data and will fill it with 0 instead.

```
# Identify the columns for creating dummy variables
cat_cols = X_train.select_dtypes('category').columns
```

```
# Create dummy variables for independent columns on training data
X_train = pd.get_dummies(X_train, columns=cat_cols, drop_first=True)
```

```
# Create dummy variables for independent columns on test data
X_test = pd.get_dummies(X_test, columns=cat_cols, drop_first=True)

X_test = X_test.reindex(columns=X_train.columns, fill_value=0)
```

5.4 Feature Scaling

5.4.1 Scale numerical features

This code is used to normalize all the numerical features so that values are only between 0 and 1. It uses Min-Max Scaling technique to do so, which rescales to a fixed range using the minimum and maximum values in the data as a reference.

1) A Min-Max scaler object is initialized, and it converts the values into a range of 0 to 1 only.
2) The scaler object is fit and transformed on the “X_train” dataset using the *.fit_transform()* function.
   The object will learn the minimum and maximum value of each column, and then transform it into the scaled value based on the formula: value = (value – min)/(max - min)
3) Similarly, the numerical features in the test data (“X_test”) are scaled. However, only the *.transform()* function is used as test data should not influence scaling.

```
# Scale the numeric features present in the training data
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Scale the numerical features present in the test data
X_test_scaled = scaler.transform(X_test)
```

3. Model Cards

Your report should include complete model cards for all candidate models, covering overview, intended use, data and features used, model configuration, performance results, and limitations or considerations.

Answer:

Model Card 1: Linear SVM Classifier

Model Overview: It is a binary classification model for predicting if a patient has cardiovascular disease or not. It separates classes by finding a hyperplane that will maximise the margin between data points. The model is *Linear* as it uses a Linear Kernel to produce class probabilities.

Intended Use:

- Binary classification of participants into "cardio=1" or "cardio=0" to support preventive measures.

Data and Features:

- The data that was fit into the model is the scaled "X_train" data, known as "X_train_scaled" and "y_train". This helps the model to learn from training data.
- Features used: "age", "gender" (dummy variables), "height", "weight", "ap_hi", "ap_lo", "cholesterol" (dummy variables), "gluc" (dummy variables), "smoke", "smoke", "alco", "active", "BMI"

Model Configuration:

- The code below shows the configuration of a Linear SVM model. A support Vector Classifier (SVC) is created and has the following attributes in it:
    - The "kernel = 'linear'" attribute is used to create a hyperplane to separate data into different classes, assuming that the data is easily linearly separable.

- The "probability = True" attribute enables probability estimates to be shown but may make the training process slower.
- The "random_state = 42" attribute ensures that the randomness is fixed and the results are reproducible every time.

- Then, the "X_train_scaled" data and "y_train" data are fit to the model, using the *.fit()* method, for the training process to start.

```
# Define and fit linear SVM
svm_model = SVC(kernel='linear', probability=True, random_state=42)
svm_model.fit(X_train_scaled, y_train)
```

- The codes below show probability estimates on test set and using a threshold. This is done to convert probabilities into predictions and then understand how the model predicts the classes.
- The SVM Model is trained on a linear kernel and the *.predict_proba()* function calculates the probability of the 2 classes in "X_test_scaled" (scaled test dataset). The "[:,1]" is used to extract all rows and selects the 2nd column which is "cardio = 1"; Essentially, it extracts the probability of HAVING cardiovascular disease, for each patient.

```
# Use predict_proba() to get the probability of positive class for all data points
svm_probs = svm_model.predict_proba(X_test_scaled)[:,1]
```

- The probabilities which are more than or equal to 0.5 will be stored as TRUE while those less than 0.5 are stored as FALSE. Then, these probabilities are converted to integer datatype using the *.astype()* function, resulting in False = 0 and True = 1. This gives the final predicted labels.

```
# Make class predictions based on default cutoff value of 0.5 on testing data
svm_pred_05 = (svm_probs >= 0.5).astype(int)
```

- The code below converts the predictions into a Series using the *pd.Series()* object and counts how many times each label appears using the *.value_counts()* function. From the output, we see that class 0 has 11894 counts where it is predicted as no cardiovascular diseases and class 1 has 8729 counts where it is predicted as cardiovascular disease.

```
# check the counts of assigned labels
pd.Series(svm_pred_05).value_counts()
```

	count
0	11894
1	8729

Performance:

A) Classification Report

- To check a small part of the performance of the 2 methods, we use *classification_report()* to give us a full outlook on precision, recall, f1-score and support. This tells us how well the model was able to perform

```
# check the performance for above two methods
print(classification_report(y_test, svm_pred_05))
print(classification_report(y_test, svm_pred_direct))
```

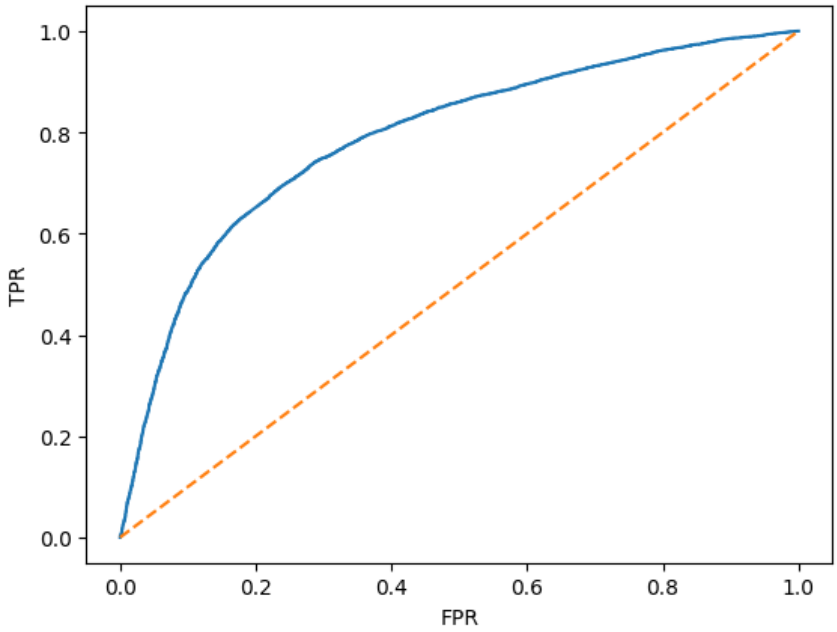
```
              precision    recall  f1-score   support

           0       0.70      0.80      0.75     10416
           1       0.76      0.65      0.70     10207

    accuracy                           0.73     20623
   macro avg       0.73      0.73      0.72     20623
weighted avg       0.73      0.73      0.73     20623

              precision    recall  f1-score   support

           0       0.69      0.83      0.75     10416
           1       0.78      0.62      0.69     10207

    accuracy                           0.73     20623
   macro avg       0.74      0.73      0.72     20623
weighted avg       0.74      0.73      0.72     20623
```

- From the above output, we see 2 different classification reports – one for SVM model that predicts based on the default cutoff value and the other is the default/direct SVM model that was initially built. We see that the overall accuracy of the model is 73%, in both reports.
- For “svm_pred_05”:
    - The report shows a more balanced approach compared to to ‘svm_direct” but tends to catch the healthy participants correctly.
    - This can be said from the high recall score of 0.8 for “cardio=0”. Hence it can be said that the model is good at identifying those who do not have any cardiovascular diseases.
    - On the other hand, the recall score for “cardio=1” is 0.65, suggesting that the model misses on 35% of actual cases.
- For “svm_pred_direct”:
    - This model can predict a little more accurately when a participant has heart disease (“cardio=1”) as the precision is higher at 0.78, compared to the first method having a value of 0.76.

- However, the recall for class 1 ("cardio=1") drops to 0.62 which means that the model is "pickier" and misses more actual heat disease cases than the previous method.

B) <u>ROC Curve</u>

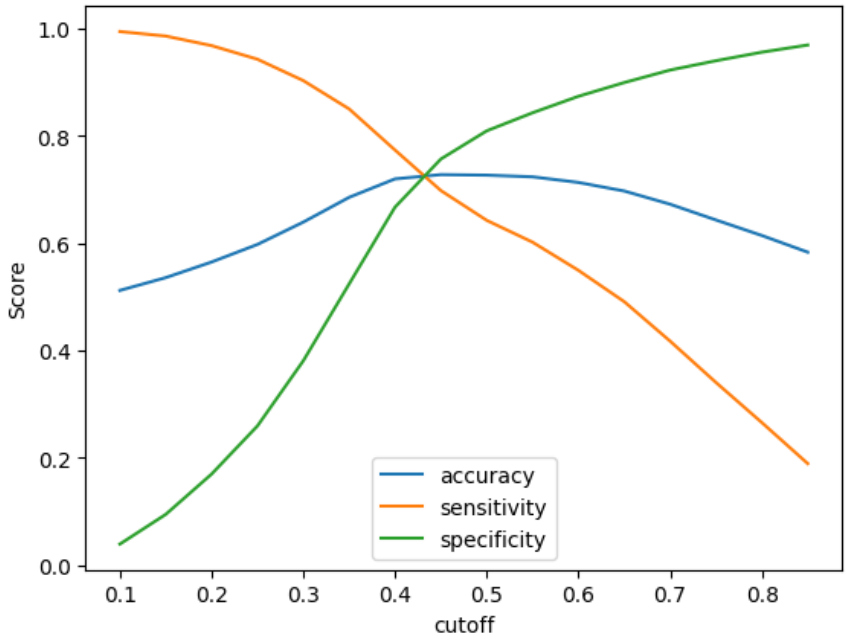
- An ROC Curve is a tool for understanding the performance of a binary classification model. It is based on 2 metrics – True positive rate, which represents the RECALL (percentage of people with cardiovascular disease that the model correctly predicted) and False positive rate, which represents FALSE POSITIVES (percentage of people without cardiovascular disease.



- From the output above, we see that there are 2 lines – Orange that represents a random guess, and Blue represents that model was able to learn meaningful patterns from the data and features that were provided to it.
- The curve shows that the model, in the beginning, was able to predict a lot of the cases easily without making many mistakes (as they can be considered as "easy"). The model then slows down and makes a few mistakes when predicting.
- We can also see that the area covered is between the range of 0.75 to 0.85 (75% to 85%) which means that the model is able to predict that a randomly chosen person has/will have a cardiovascular disease more often than predicting that a randomly chosen person does not/will not have a cardiovascular disease by 85%.

C) <u>Precision-Recall Curve</u>

- A precision-recall curve shows the positive predictive value vs sensitivity at different thresholds, where predicting a positive class is more important than accuracy of the model.



- From the output above, we see that accuracy, represented by a blue line, is the percentage of total correct predictions. It peaks around 0.5, which implies that this threshold is good for general performance of the model.
- We can see that sensitivity/recall, represented by an orange line, is the highest at low cutoffs. As the cutoff increases, the model makes mistakes and misses actual cardiovascular disease cases. We can also see that specificity, represented by the green line, is used to measure how well a model correctly identifies true negative cases.
- The intersection between sensitivity and specificity, at a cutoff of 0.42, is considered as the optimal balance point. At this point, the model is equally likely to falsely identify a patient with the disease and miss the patient with the disease.

Limitations & Considerations:

- The model needs feature scaling as SVM is sensitive to the feature values. If feature scaling was not done, it could lead to some of the features dominating over other features and cause the model to decide on incorrect hyperplanes.
- Support Vector Machine (SVM) model is generally difficult to interpret compared to the other simpler models like Decision Trees. As SVM creates a hyperplane, it makes it difficult to understand how specific features may influence predictions that occur regarding patients having or not having cardiovascular disease.
- There could also be some errors where false positives from the model could lead to further unnecessary examinations in the hospital or missing a high-risk patient who needs an immediate examination.

- Another consideration is that there could be some domain-specific cautions where it would be best to process all input features and scale them in the same manner during the model training.

Model Card 2: Decision Tree Classifier

Model overview: A decision tree classifier is known for its ease of understanding and is generally used for binary classification

Intended Use:

- Binary classification of cardiovascular disease risk, where 0 = no cardiovascular disease and 1 = cardiovascular disease.
- To help healthcare professionals to understand the risks factors that are critical and allow decision-making processes to take place.

Data and Features:

- Raw features from original dataset: “gender”, “ap_hi”, “ap_lo”, “cholesterol”, “gluc”, “smoke”, “alco”, “active”, “cardio”
- Transformed features: “BMI” (following the formula involving height and weight), “age” (converted to years), “weight” (converted to kg), “height” (converted to meters)

Model Configuration:

- Defining a decision tree classifier
    - A *DecisionTreeClassifier()* object is created with the attribute “random_state=42” so that the randomness is fixed and the result is reproducible every time.
    - The *.fit()* function fits the scaled training data (“X_train_scaled”) and the target training data (“y_train”)

```
# Define and fit
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train_scaled, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

- Get the feature importance scores from the trained model

- A Series is created using *pd.Series()* with the *.feature_importances_* in-built attribute being used to get the importance score of each feature in making predictions. Each value represents how much it contributed to predicting cardiovascular disease, hence, if the value is higher, it means that the feature is more important.
- The feature names are then assigned to the scores using "index = X_train.columns" and then, these values are sorted in descending order using *.sort_values(ascending=False)*
- From the output, we can see that "age" is the most important feature in decision-making (it contributes to 28.4% of the process). Overall,
  the strong predictors are: "age", "ap_hi" and "BMI"
  the moderate predictors are: "height", "weight", "ap_lo", "cholesterol"
  the weak predictors are: "gender", "active" (physical activity)

```
importance = pd.Series(
    dt_model.feature_importances_,
    index=X_train.columns
).sort_values(ascending=False)

importance.head(10)
```

age	0.284312
ap_hi	0.226804
BMI	0.158267
height	0.097321
weight	0.095044
ap_lo	0.038403
cholesterol_2	0.022385
gender_1	0.017244
active_1	0.013622
cholesterol_1	0.011834

Performance:

Before diving into different sections, the classification report and ROC, precision-recall and sensitivity-specificity tradeoff curves are based on the optimal cutoff that was found to be more than 0.4 to approximately 0.5. The following code filters the probabilities based on the condition mentioned above so that it can be used for evaluation of the model.

```
dt_final_test_pred = (dt_probs >= 0.4).astype(int)
dt_final_train_pred = (
    dt_model.predict_proba(X_train_scaled)[:,1] >= 0.4).astype(int)
```

A) <u>Classification Report</u>

- The classification report below shows the different types of metrics that are used – precision, recall, f1-score, accuracy and support

- The metrics below show the performance of the model on the test dataset. Class 0 means No cardiovascular disease and based on the metrics; we can understand that the model can detect participants with no cardiovascular disease reasonably but not accurately.
    - Precision = 0.63; This means 63% of predicted non-cardiovascular patients (cardio = 0) are non-cardiovascular patients.
    - Recall = 0.64; This means that 64% of participants were correctly identified as non-cardiovascular patients.
    - F1-score = 0.63; This means that there is a balanced performance of the model between precision and recall
    - Accuracy = 0.63; This means that the model can accurately predict participants with or without cardiovascular disease only 63% of the time, suggesting that there is room for improvement in the model's training.

Test Performance

	precision	recall	f1-score	support
0	0.63	0.64	0.63	10416
1	0.63	0.62	0.63	10207
accuracy			0.63	20623
macro avg	0.63	0.63	0.63	20623
weighted avg	0.63	0.63	0.63	20623

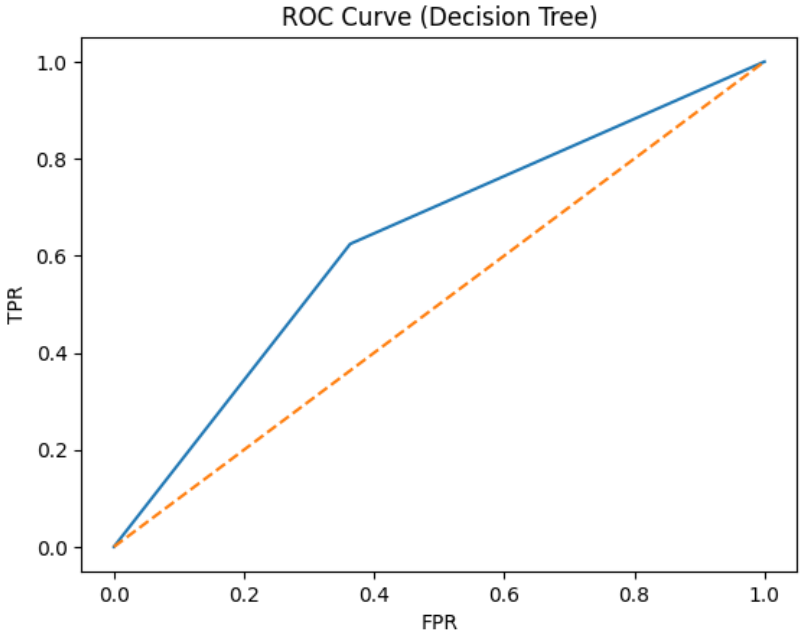
- The metrics below show the performance of the model on the train dataset. From the output, we can see that all the metrics (precision, recall, f1, accuracy) are 1.00 (100%). This means that the model assumes that there are no false positives and false negatives that exist in the dataset
- This means that the model can make perfect predictions on the training data, indicating overfitting since in the real-world, achieving perfect accuracy is rare. This could have happened as decision trees do not have any restrictions; hence it learns every small detail, captures noise and creates complex structures.

Train Performance

	precision	recall	f1-score	support
0	1.00	1.00	1.00	24303
1	1.00	1.00	1.00	23816
accuracy			1.00	48119
macro avg	1.00	1.00	1.00	48119
weighted avg	1.00	1.00	1.00	48119

B) <u>ROC Curve</u>

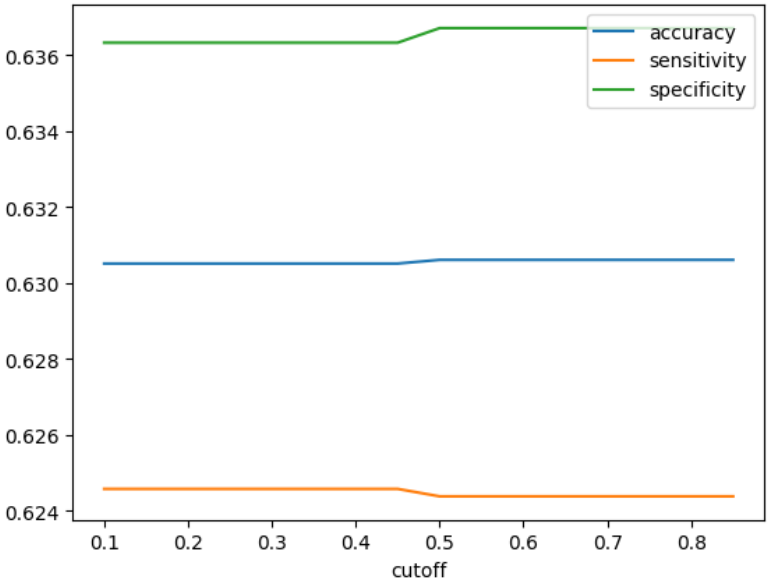
- Like the ROC curve for the other model, the y-axis represents the true positive rate (TPR) which is the proportion of actual positives the model correctly identified, and the x-axis represents the false positive rate (FPR) which is the proportion of actual negatives that the model predicted as false positives.
- From the graph, we can approximate that FPR is 0.36 and TPR is 0.62. We also see an orange line which represents a random classifier. Since the blue line (model's performance) is above the orange line, it means that the model is able to predict rather than guessing but not by much.



C) <u>Sensitivity-Specificity Tradeoff</u>

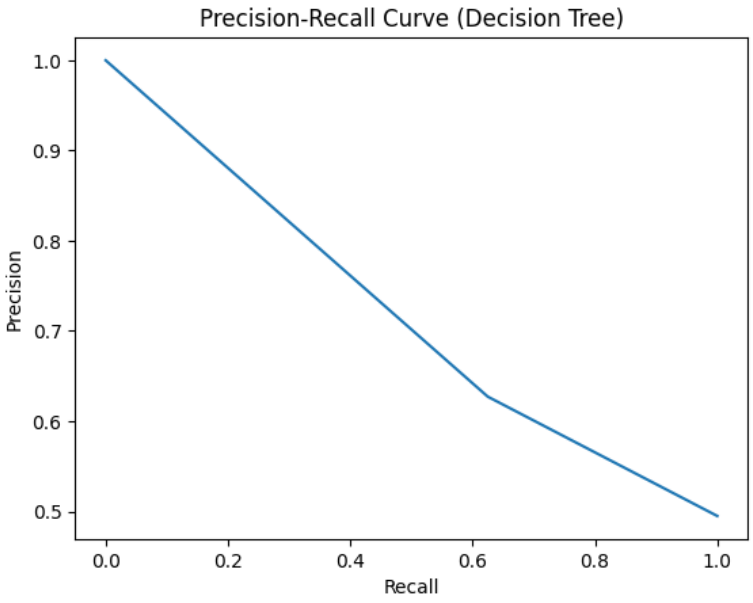
- From the values obtained and the graph given below, we see that it follows a "flatline" where the lines are almost horizontal.
- The reason for this happening is because decision trees do not produce a class distribution of probabilities. We see that from threshold = 0.1 to threshold = 0.4, there is no change in accuracy, sensitivity and specificity – as represented by the 3 lines in the graph below as well.
- At threshold = 0.5, there is a small change in the values and as depicted in the graph too.
    - Accuracy and specificity have a small "step" upwards while sensitivity has a small step "downwards".
    - This could suggest that only a small cluster of data points have their predicted probability at threshold = 0.5, but do not exist at the other threshold values below and above 0.5 respectively.

	cutoff	accuracy	sensitivity	specificity
0	0.10	0.630510	0.624571	0.636329
1	0.15	0.630510	0.624571	0.636329
2	0.20	0.630510	0.624571	0.636329
3	0.25	0.630510	0.624571	0.636329
4	0.30	0.630510	0.624571	0.636329
5	0.35	0.630510	0.624571	0.636329
6	0.40	0.630510	0.624571	0.636329
7	0.45	0.630510	0.624571	0.636329
8	0.50	0.630607	0.624375	0.636713
9	0.55	0.630607	0.624375	0.636713
10	0.60	0.630607	0.624375	0.636713
11	0.65	0.630607	0.624375	0.636713
12	0.70	0.630607	0.624375	0.636713
13	0.75	0.630607	0.624375	0.636713
14	0.80	0.630607	0.624375	0.636713
15	0.85	0.630607	0.624375	0.636713



D) <u>Precision-Recall Curve</u>

- From the graph below, we see that the model has a 100% performance based on the metrics for accuracy, precision and recall. This means that the tree grew deep (without pruning) and memorized every datapoint, leading to the model overfitting on the test data.
- We can see that precision decreases as recall increases. When recall is low, the precision is 1 (100%) but as recall increases, the precision has been dropping. This suggests that as the model tries to detect more cardiovascular patients, it has been detecting some false positives, causing the precision to decrease.
- Finally, we can assume that the decision tree classifier did not perform well on this dataset due to the steep drop of the curve. A good precision-recall curve should constantly stay high and curve upwards to represent that as the precision is high, the recall will also be high. Here however, the precision drops quickly and there is almost no curve present.



Limitations & Considerations:

- Decision trees are prone to overfitting and poor generalization to unseen data as they memorize the data by growing till the maximum depth. This is also shown in the report where the model achieved 100% accuracy on training data but only 63% on the test data.
- They are also sensitive and small changes in data, especially in the important features, can lead to a different tree structure with different predictions.
- Since decision trees can grow deep leading to overfitting, they require hyperparameter tuning to improve performance and make the model less complex.
- There might be some biased feature importance due to the splits that occur in decision tree classifiers. This may cause some of the features to be exaggerated as important when they are not.

Model Card 3: RBF Support Vector Machine

Model Overview: It is a non-linear classification model used to separate data points in a higher-dimensional space.

Intended Use:

- Use for binary classification of cardiovascular disease based on common health indicators.
- Can be used on data where they tend to have non-linear relationships like blood pressure (“ap_hi” and “ap_lo”), cholesterol, etc.

Data and Features:

- Input features: “age”, “gender”, “height”, “weight”, “BMI”, “ap_hi”, “ap_lo”, “cholesterol”, “gluc”, “smoke”, “alco”, “active”
- Target variable: “cardio” (0 or 1)

Model Configuration:

- Defining a SVM model with RBF Kernel
  - RBF stands for radial basis function, and it is a non-linear kernel used to capture complex relationships between features and then map data into a higher-dimensional space for a curved decision boundary.
  - Like the Linear SVM model, most of the configuration is the same except for the kernel which has now been changed to RBF.

```
svc_rbf = SVC(kernel='rbf', probability=True, random_state=42)
svc_rbf.fit(X_train_scaled, y_train)
```

```
SVC
SVC(probability=True, random_state=42)
```

Performance:

- The model has been evaluated on “y_test” and scaled “X_test” data, using the *.predict()* function.
- From the output below, we can see that there is a small improvement in the model’s performance, based on the accuracy on the test data – 73%, compared to the Linear SVM’s accuracy at 72%. This suggests that the RBF SVM can capture more complex correlations than the Linear SVM which led to the improvement of accuracy of the model.
- Like the accuracy, the precision and recall values have also increased, compared to the linear SVM
  - The precision increased from 0.69, in linear SVM, to 0.76. This means that the RBF SVM can predict more accurately and has reduced false positives.
  - The recall for “cardio=0” (non-cardiovascular disease patients) increased from 0.67, in linear SVM to 0.8. This means that the RBF SVM can correctly identify 80% of non-cardiovascular disease patients and reduce the false predictions (both false positives and false negatives).
  - However, the recall for “cardio=1” (patients with cardiovascular disease) is better in the linear SVM model (0.77) than the RBF SVM (0.66). This means that

the linear SVM can predict cardiovascular disease better than the RBF SVM model.

- Overall, the RBF SVM model has a balanced performance compared to the linear SVM as it is able to maintain good precision, recall and accuracy.

	precision	recall	f1-score	support
0	0.70	0.80	0.75	10416
1	0.76	0.66	0.71	10207
accuracy			0.73	20623
macro avg	0.73	0.73	0.73	20623
weighted avg	0.73	0.73	0.73	20623

Limitations & Considerations:

- RBF Kernel depends on parameters which means that they should be carefully tuned else it can lead to underfit or overfit on test data.
- RBF SVMs are harder to interpret as they do not have clear decision rules to understand how predictions are made, like decision trees.
- This form of SVM is computationally expensive, takes more time to train on the vast amount of data and then make predictions on the test data, compared to linear SVMs and decision trees.

Model Card 4: Hyperparameter Tuning of Decision Tree (GridSearchCV)

Model Overview:

Intended Use:

Data and Features:

Model Configuration:

Performance:

Limitations & Considerations:

---

4. Final Conclusion

Your report should summarise key insights from all stages: important findings from features and EDA, baseline vs tuned model performance, overfitting issues and how they were

resolved, assessment of data sufficiency, the suitability of linear vs non-linear models, the reasoning behind the final model choice, and the final outcomes of the modelling process.

Answer:

Question 1: What insights can we gain from exploring the relationships between different health metrics and the prevalence of cardiovascular disease within the patient population?

- We can see that cardiovascular disease occurs among middle-aged to older individuals who are within the age range of 50 to 60 years. This implies that "age" is one of the important features and risk factors.
- From some of the results, we can also see that high blood pressure ("ap_hi", "ap_lo") show a strong correlation with cardiovascular disease, where the higher the blood pressure, the riskier it is to get cardiovascular disease.
- Similarly, weight and BMI also affect the risk of getting cardiovascular disease. The higher they are, the higher the risk of getting cardiovascular disease.
- This also shows that there are multiple factors that can influence cardiovascular disease to occur in people, as opposed to only 1 most important/common factor.
- Height has little to no significant effect on the correlation with cardiovascular disease, hence it can be said that it is a weak predictor.

Question 2: Based on the analysis, which patient characteristics emerge as the strongest predictors of cardiovascular disease risk? Are there any surprising or unexpected findings?

- Strongest predictors: Age, BMI, Cholesterol, Glucose, Weight, Physical Activity ("active"), Systolic blood pressure ("ap_hi"), Diastolic blood pressure ("ap_lo").
- "Cholesterol" had shown a very strong relationship with "cardio" which also aligns with the research done on it where high cholesterol means increased risk of getting cardiovascular disease.
- Smokers ("smoke") and alcohol consumption ("alco") showed low cardiovascular disease probability which is the opposite of what common medical knowledge says. This could imply that there has been some underreporting in the dataset by the patients, since it contradicts general medical advice.
- Overall, the "cholesterol", "ap_hi" and "ap_lo" (blood pressrure) are the most important features in the dataset. These 3 features can also be considered as major factors for cardiovascular disease in a person.

Question 3: How effectively can machine learning models identify individuals at risk of developing cardiovascular disease based on their health data? How does the evaluation results vary across different models?

- The models, linear SVM, RBF SVM (support vector machine) and decision trees were able to successfully predict cardiovascular disease with accuracy in the range of 63%-73%.
- We can see that the linear SVM model has good generalisation to unseen and new data, showing a balanced performance with stable results.
- The RBF SVM model is better at capturing non-linear relationships with a slightly improved accuracy. However, there is an increased computational cost (resources and time).
- On the other hand, the decision tree has a high training accuracy but has poor training accuracy and has an issue of overfitting on the data.

Question 4: How can CardioCare integrate the predictive model into their existing healthcare workflows to enhance preventative care strategies?

- CardioCare can integrate the predictive model into the electronic record systems that are already present so that the model can be used for early screening of the risk of cardiovascular disease. This can help in automatically flagging high-risk patients during their checkups so that doctors can speed up their diagnosis.
- Other than this, risk scores and alerts can be provided to doctors to help them assist in figuring out which patients needed preventive care and recommend some lifestyle changes to them.
- Real-time monitoring on patients' health can also be implemented to reduce the burden that healthcare professionals face by focusing more on high-risk heart disease individuals. This in turn will help improve overall efficiency of healthcare and decision-making between doctors.
- Finally, dashboards can be used to display patient risk levels accurately and visually. This can be integrated into e-record systems or mobile applications so that patients and doctors can continuously monitor their health.
